Data Frames de Pandas

Estadística con Python

Mario A. Morales Rivera
mamorales@correo.unicordoba.edu.co
Universidad de Córdoba

2026-09-21



Tabla de contenidos I

1. Pandas DataFrames
2. Creación y operaciones con `DataFrames` de pandas
3. Inspeccionando `Data Frames`
4. Creación a partir de fuentes externas
5. Operaciones con `DataFrames`
6. Filtrado de datos con pandas
7. Unir Marcos de datos de Pandas



Pandas DataFrames



Pandas DataFrames

Marcos de datos de Pandas

Nota

El marco de datos de Pandas o *Pandas `DataFrame`* es la estructura principal de datos en esta biblioteca de Python. Un marco de datos de Pandas es un arreglo en forma de tabla (bidimensional) que soporta columnas de (potencialmente) tipos heterogéneos de datos.

Marcos de datos de Pandas (II)

- Los `DataFrame` son de tamaño mutable, es decir, se les puede agregar o quitar filas o columnas después de su creación;
- Sus filas y columnas (conocidos como ejes, *axis*) están etiquetados y se pueden efectuar operaciones aritméticas teniendo en cuenta dichas etiquetas.
- Los `DataFrames` de Pandas se pueden considerar como un contenedor similar a un diccionario para objetos que son `Pandas Series`.



Creación y operaciones con `DataFrames` de pandas



Creación y operaciones con DataFrames de pandas

Creación de Pandas DataFrames

- A continuación se crea un DataFrame de Pandas.
- Lo primero que hacemos es importar el paquete *Pandas* mediante `import pandas as pd`.
- Como se dijo en la clase anterior, no es obligatorio que se importe con el nombre pd, pero es lo usual en la comunidad de usuarios y útil para compatibilidad del código.
- Luego creamos un DataFrame de nombre df, con tres columnas y siete filas.

Creación de DataFrames

```
import pandas as pd
import numpy  as np
df=pd.DataFrame({"col1":[1,3,2,4,7,9,5],
                 "col2":[9,0,2,1,7,3,4],
                 "col3":["F","M","F","F","F","M","M"]})
```

- Nota: en la función DataFrame tomamos como argumento un diccionario cuyas

```
d={'A': [1, 2, 3], 'B': [4, 5, 6]}                    ①
dfd=pd.DataFrame.from_dict(d,orient='index')          ②
dfd.columns=["uno","dos","tres"]                      ③
```

① Diccionario con listas como valores.
② La opción `orient='index'` hace que los valores del diccionario, que son las listas, pasen a ser las filas del marco de datos.
③ Asignamos nombres a las columnas. También podemos hacerlo mediante, `columns=["uno","dos","tres"]`, dentro del constructor `from_dict`.



Inspeccionando `Data Frames`



Inspeccionando Data Frames

Inspeccionando Data Frames

Con los métodos `.head(k)` y `.tail(k)` podemos examinar las k primeras y últimas filas del `DataFrame`, respectivamente.

```
df.head(4) # 4 primeras filas
```

```
   col1  col2 col3
A     1     9    F
B     3     0    M
C     2     2    F
D     4     1    F
```

```
df.tail(3) # 3 últimas filas
```

```
   col1  col2 col3
E     7     7    F
F     9     3    M
G     5     4    M
```



Creación a partir de fuentes externas



Creación a partir de fuentes externas

Lectura de datos desde archivos externos.

Es posible crear `Pandas DataFrames` a partir de datos que se encuentran en fuentes externas, tales como archivos de `MS Excel` (`.xls` o `.xlsx`), Open Document Format (`.odf`), ficheros de texto plano (`.txt` o `.csv`), en bases de datos relacionales `SQL` e incluso en tablas de páginas web. La lectura de datos en el formato correcto y verificar que no hayan errores en el proceso, es la parte que toma más tiempo en el análisis de datos.

Lectura de datos desde archivos externos (II)

Cuando los datos están organizados en un archivo de texto plano, deberíamos iniciar siempre con una inspección visual de los datos, en particular deberíamos chequear:

- ¿Tienen los datos un encabezado al inicio o texto al final?
- Lo usual es que el encabezado tenga los nombres de las columnas,
- Las notas al final de los archivos se usan para dar una descripción de los datos.

Por ejemplo, se explica el contenido de las columnas, si hay alguna codificación de las respuestas que haya que tener en cuenta, entre otras cosas.



Operaciones con `DataFrames`



Operaciones con DataFrames

Selección, adición y borrado de columnas.

- Con `df['x']` podemos seleccionar la columna de nombre `x`, del Pandas DataFrame, `df`.
- De manera equivalente, mediante `df.x`.
- A continuación seleccionamos `col1` del marco de datos `df`

```
df['col1']
```

```
A    1
B    3
C    2
D    4
E    7
F    9
G    5
Name: col1, dtype: int64
```



Selección, adición y borrado de columnas (II)

Filtrado de datos con pandas



Filtrado de datos con pandas

Selección de filas y columnas con `iloc` y `loc`

- El método `iloc` selecciona tanto filas como columnas basado en la posición del elemento.
- El formato de la sintaxis es: `df.iloc[<fila>,<columna>]`
- `fila` y/o `columna` corresponde a un número entero o una lista de enteros que indican la posición (índice) de las filas o columnas a seleccionar.
- Esta forma de seleccionar filas con `.iloc` también soporta índices negativos, en cuyo caso la selección se realiza empezando por la última fila (o columna).
- Cuando se entrega un solo valor, por ejemplo `df.iloc[valor]` la selección se hace sobre las filas, veamos su funcionamiento con ejemplos concretos.

Selección de filas y columnas con `iloc` y `loc` (II)

```
df.iloc[0:3]      ①
df.iloc[0]        ②
df.iloc[[0,4,6]]  ③
```



Unir Marcos de datos de Pandas



Unir Marcos de datos de Pandas

Unir marcos de datos de Pandas

En caso que se tengan dos marcos de datos con exactamente las mismas columnas, por ejemplo, cuando tenemos información de hombres y mujeres separada en objetos distintos, podríamos unir los dos en uno, con la función `concat`.

Marcos de datos para unir

- Para ilustrar el uso de `concat` crearemos dos marcos de datos, `dfM` y `dfF` que contienen datos de edad, estatura, y género.
- Note que todas las filas de `dfM` tienen `Genero=M` y todas las de `dfF`, tienen `Genero=F`.

```
dfM=pd.DataFrame({"Edad":[24,21,18,26],
     "Estat":[1.65,1.72,1.69,1.8],
     "Genero":["M","M","M","M"]})
dfF=pd.DataFrame({"Estat":[1.60,1.58,1.62,1.71,1.64],
"Genero":["F","F","F","F","F"],
"Edad":[23,22,19,25,21]})
```